

MSE 312: MECHATRONICS DESIGN II

Integration Lab 1: Using Simulink to Develop and Run Standalone Algorithms on Target Embedded Hardware

OBJECTIVES

To configure and utilize Matlab/Simulink for developing and running standalone data acquisition and control algorithms on target embedded hardware.

Required Components/Material:

- Matlab/Simulink 2023a (some toolboxes may not work with newer versions)
- MEGA 2560 R3 Board ATmega2560 ATMEGA16U2 + USB Cable for Arduino
- Breadboard, LED, 1 k Ω Resistor, 10 k Ω Resistor, Arduino Jumper Cables, Button Micro Switch, 10 k Ω Potentiometer

References: Arduino Support from MATLAB/Simulink (www.mathworks.com)

PROCEDURE

ACTIVITY 1.1: Connecting Matlab & Arduino

Matlab® Support Package for Arduino® Hardware enables you to use Matlab to communicate with the Arduino® board over a USB cable. With this type of communication, the board “listens” to commands arriving via its serial port, executes the commands, and, if needed, returns a result. This approach helps you:

- Work in the Matlab shell for interactive development and debugging.
- Interactively develop programs to acquire analog and digital data, and to control DC, servo, and stepper motors.
- Access peripheral devices and sensors (e.g., connected over I2C or SPI).
- Run control loops at up to 25 Hz (not real time).

To connect the Arduino board with Matlab through a USB cable, the Matlab Arduino support package needs to be installed. If it is not installed (e.g., you are using your own computer), do the following (or ask the System Administrator- if not installed on lab computers):

1. Open Add-Ons Explorer in Matlab
2. Choose MATLAB Support Package for Arduino Hardware
3. Click install and follow the steps to download and install support packages (Matlab and third party)
4. Connect Arduino to host computer via USB
5. When asked for board selection, choose Mega2560 and a COM port (USB interface)
6. Select required libraries (make sure to include RotaryEncoder, for this lab, this is the only library you will need)

Example 1.1: Test the communication

To check if the board is recognized in Matlab, run the following command:

```
>> a=arduino()
```

You should see the details of the board and get a response similar to the following

```
a = arduino with properties:
    Port: 'COM3'
    Board: 'Mega2560'
    AvailablePins: {'D2-D53', 'A0-A15'}
    AvailableDigitalPins: {'D2-D53', 'A0-A15'}
    AvailablePWMPins: {'D2-D13', 'D44-D46'}
    AvailableAnalogPins: {'A0-A15'}
    AvailableI2CBusIDs: [0]
    AvailableSerialPortIDs: [1, 2, 3]
    Libraries: {'I2C', 'SPI', 'Servo'}
```

This serves as a connection test for the hardware. It shows which serial port the Arduino is connected to, “COM3” (note this down for later in this exercise), which analog/digital pins are available, and which libraries are loaded. If any of the necessary libraries are missing, or the board type or com port is not the correct one, you may get an error message. If you have already created an Arduino object and want to change some of these properties, make sure to clear the previous one using the following command before creating a new object with the desired properties (otherwise you will get an error message):

```
>> clear a
```

You can use the following command to specify port, board, and libraries:

```
>> a = arduino('COM3', 'Mega2560', 'Libraries', {'I2C', 'RotaryEncoder', 'SPI', 'Servo', 'ShiftRegister'})
```

Example 1.2: Test the digital I/O

1. Choose a digital pin to write to (e.g. D11). Connect it to the anode (positive) pin of an LED and use a 1 k Ω resistor to connect the cathode (negative) pin of the LED to the GND (ground) as indicated in the figure below

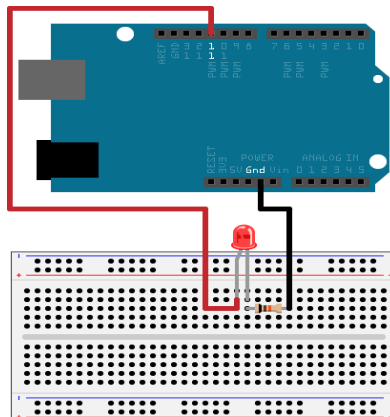


Figure 1.1: Test circuit for example 1.2.

2. Write the following commands in Matlab and analyze the results

```
>> writeDigitalPin(a, 'D11', 1);
```

```
>> writeDigitalPin(a, 'D11', 0);
```

3. After setting the digital output pin D11 to High, measure the maximum voltage of the digital output pin with a multimeter.
4. Once the connection is no longer needed, clear the Arduino object as follows:

ACTIVITY 1.2: Connecting Simulink & Arduino

The “Simulink Support Package for Arduino” enables you to use Simulink to develop and run standalone algorithms on target embedded hardware (Arduino). This package can be used to do the following:

- Develop and simulate your algorithms in Simulink and use automatic code generation to run them on the target device (Arduino, etc.)
- Incorporate signal processing, control design, state logic, and other advanced math and engineering routines in your hardware projects
- Interactively tune and optimize parameters as your algorithm runs on the device

To use Simulink, the “Simulink Support Package for Arduino Hardware” needs to be installed, if not installed, (e.g., you are using your own computer), do the following (or ask the System Administrator- if not installed on lab computers):

1. Open Add-Ons Explorer in Matlab
2. Choose “Simulink Support Package for Arduino Hardware”. If you cannot find it, it might be found under other names such as “Legacy Matlab and Simulink Support for Arduino”, or “Embedded Coder Target for Arduino” (this might happen if the Matlab version is different from the recommended one).
3. Click install and follow the steps to download and install support packages (Matlab and third party)

To access the Simulink blocks used to program the Arduino hardware, do the following:

1. Clear your workspace in Matlab (Matlab syntax: *clear all*). If the Arduino object, defined in the previous activity, is not cleared from the workspace, it may cause problems due to Matlab trying to create two connections with the same hardware.
2. Open “Simulink”, click the “Blank Model” template, and click Simulink Library Browser
3. Click “Simulink Library Browser”. From the left side options, click Simulink Support Package for Arduino Hardware as illustrated below

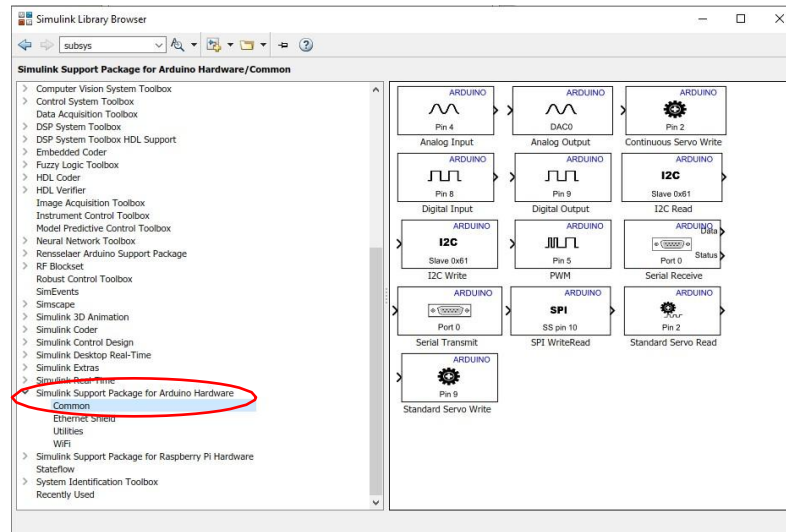


Figure 1.2: Simulink blocks used to program the Arduino hardware.

Example 1.3: Read analog input and display

1. Use Arduino jumper cables to connect the 5V and GND pins of the Arduino board to the breadboard to use it as a voltage source. Connect the VCC and GND pins of the potentiometer to the board, and connect the output of the potentiometer to an arbitrary analog input pin of the Arduino (e.g., Pin A0).

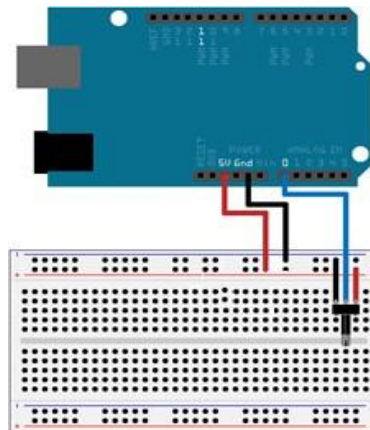


Figure 1.3: Test circuit for example 1.3.

2. Drag and drop the following two blocks to the new Simulink model you have created: “analog input” from *Simulink Support Package for Arduino Hardware >>common*, and “Scope” from *Simulink>>sinks*
3. Choose the Arduino’s analog input pin number that you want to read from (e.g., pin A0) and your preferred sampling time (e.g. 0.01s) in the analog input block parameters and connect the block to the scope.

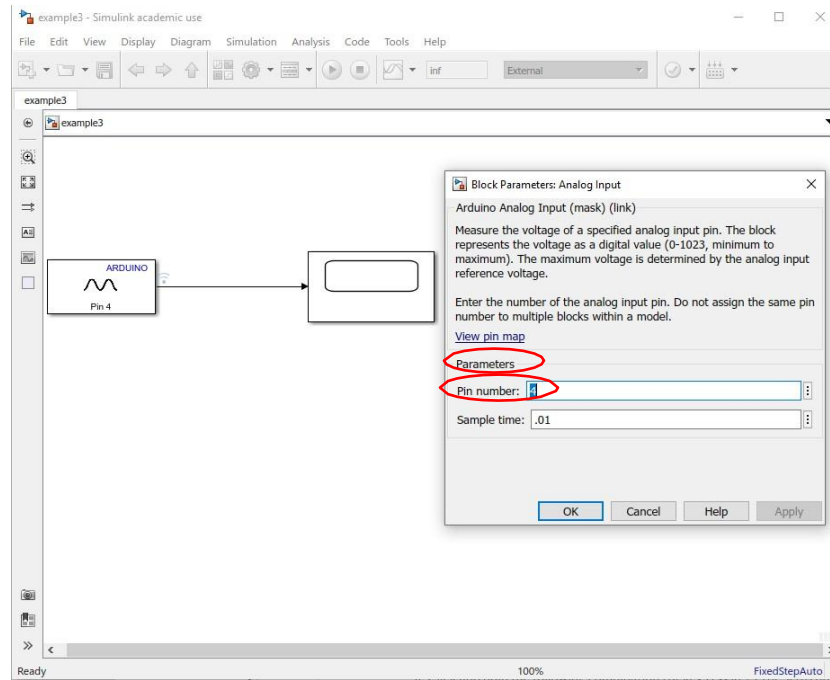


Figure 1.4: Block parameter settings for Arduino analog input.

4. From the gear icon (click on “Modeling” menu) on top of the Simulink model, choose the model configuration parameters option as follows:

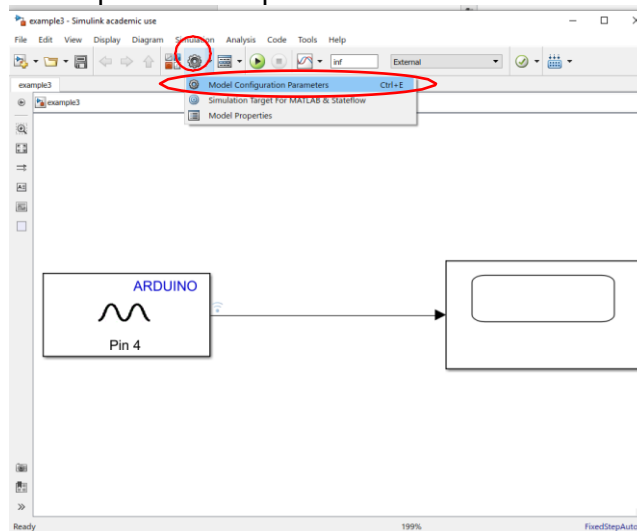


Figure 1.5: Accessing model configuration parameters.

5. From the configuration parameters window that opens, choose hardware implementation on the left hand side menu and select the Arduino Mega 2560 as the hardware board:

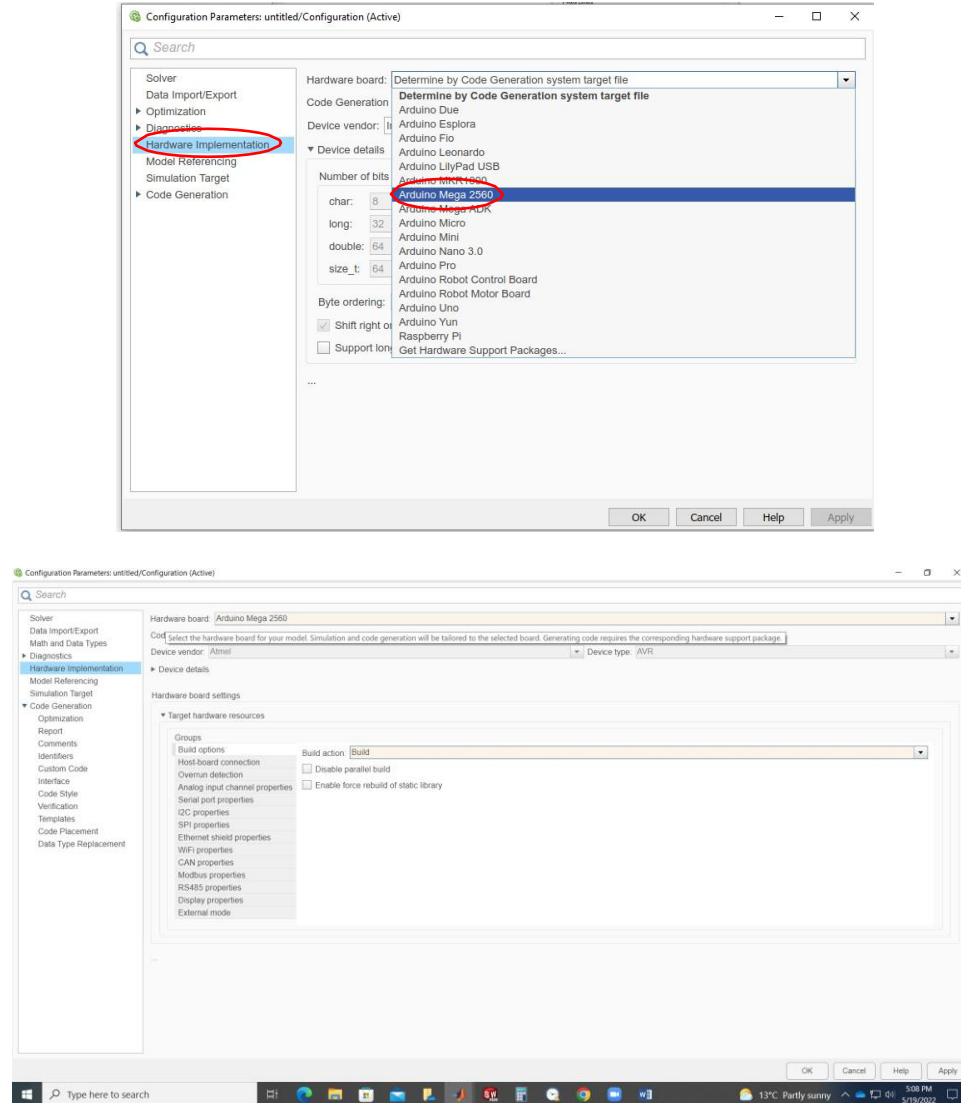


Figure 1.6: Hardware Implementation settings.

After choosing the Mega 2560 as your Hardware board, the Target hardware resources option appears at the bottom of the window. From Groups, select Host-board connection and choose the set host COM port as manually. Insert the COM port number you got while testing the communication in example 1.1 (e.g., COM 3). Click OK to apply the changes.

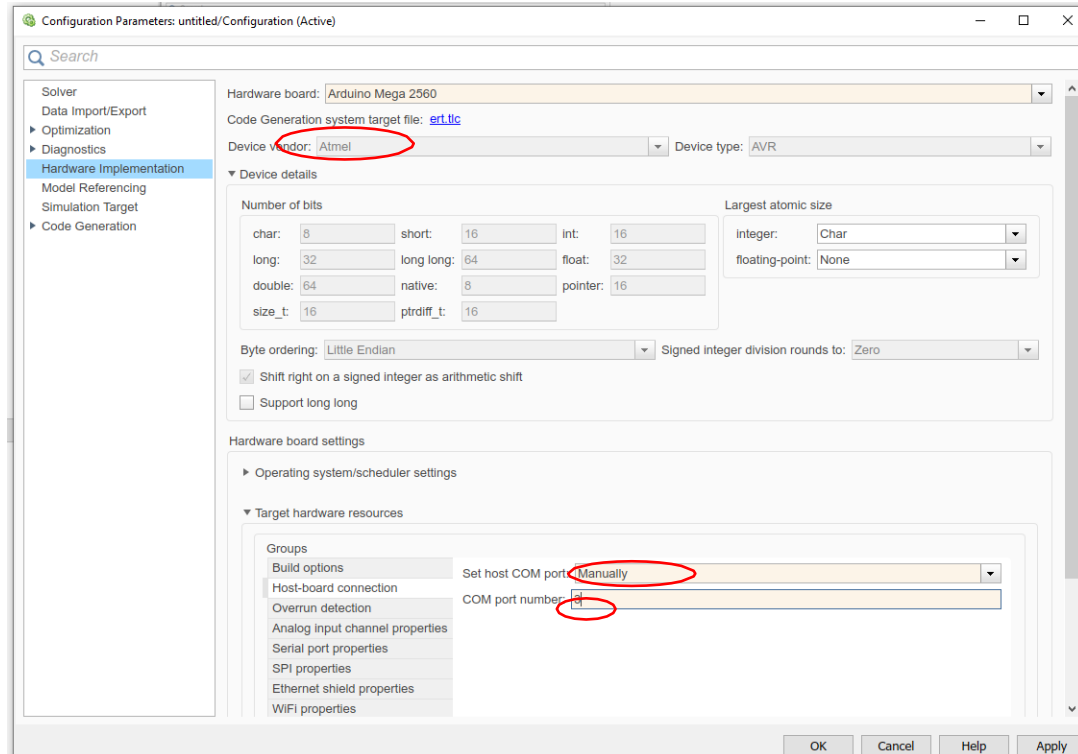


Figure 1.7: Host-board connection settings.

6. In the the Solver tab, for the Start time field, enter 0. In the Stop time field, enter the amount of time you want your model to run. For example, enter inf if you want to run the program indefinitely. From the Type list, choose Fixed-step. From the Solver list, choose a solver. For example, ode5 (Dormand-Prince). In the Fixed step size field, enter a sample time. For example, enter 0.01 seconds for a sample rate of 100 samples/second . This sample time should be the same as the sample time in your blocks. **It is recommended to keep the sample time constant in all blocks.**

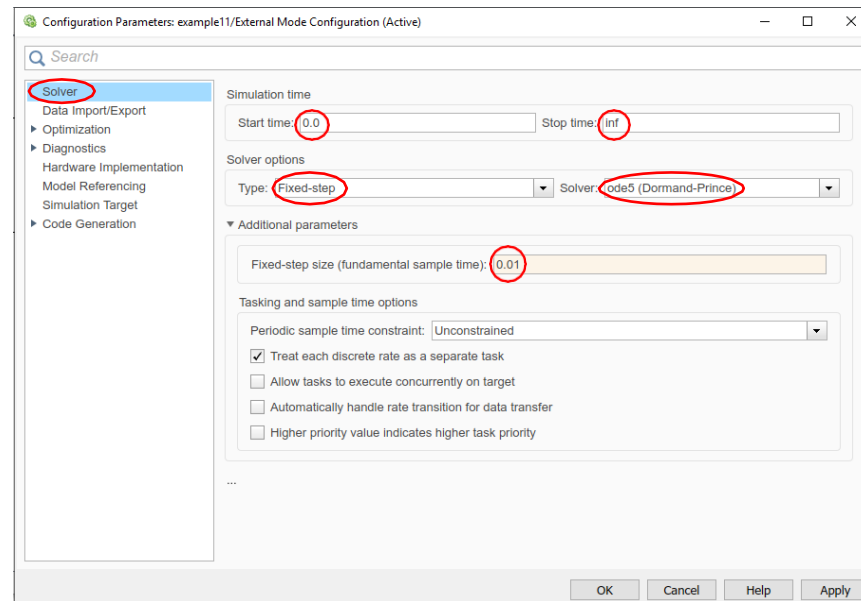


Figure 1.8: Solver settings.

7. In the Simulink model window, choose the simulation mode as External (Run on Board: either of the following figures depending on the Simulink version). Save the model as “example3.slx” and run the simulation.

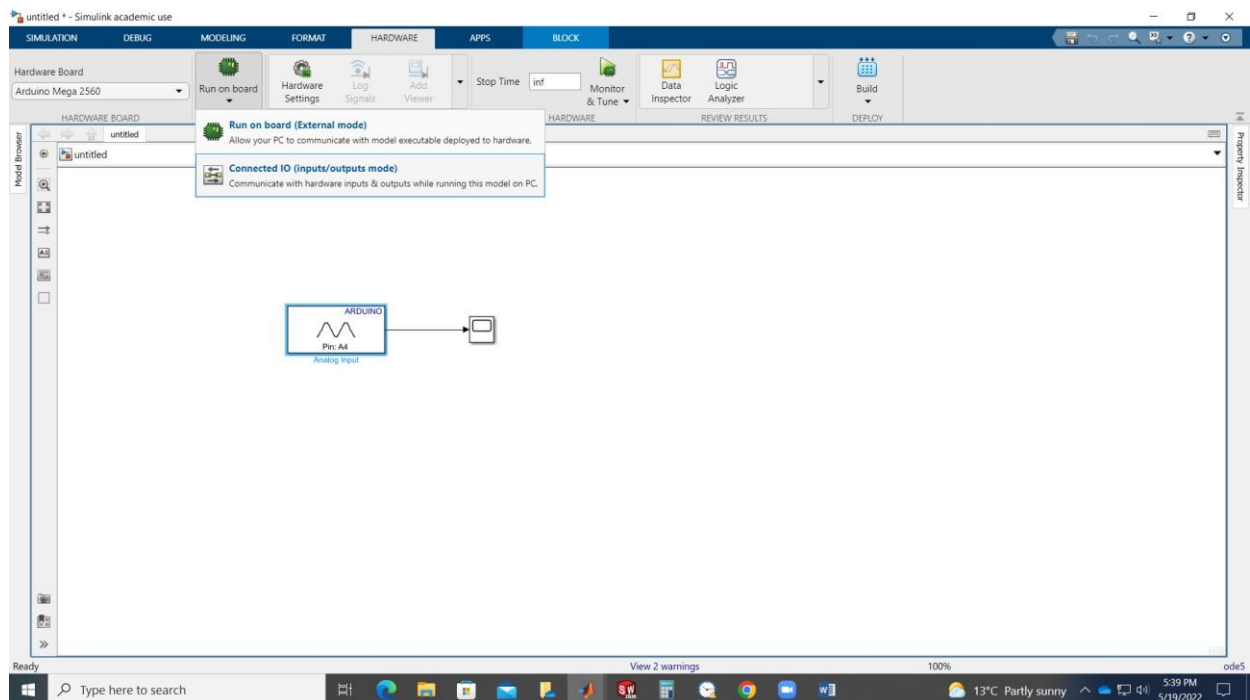
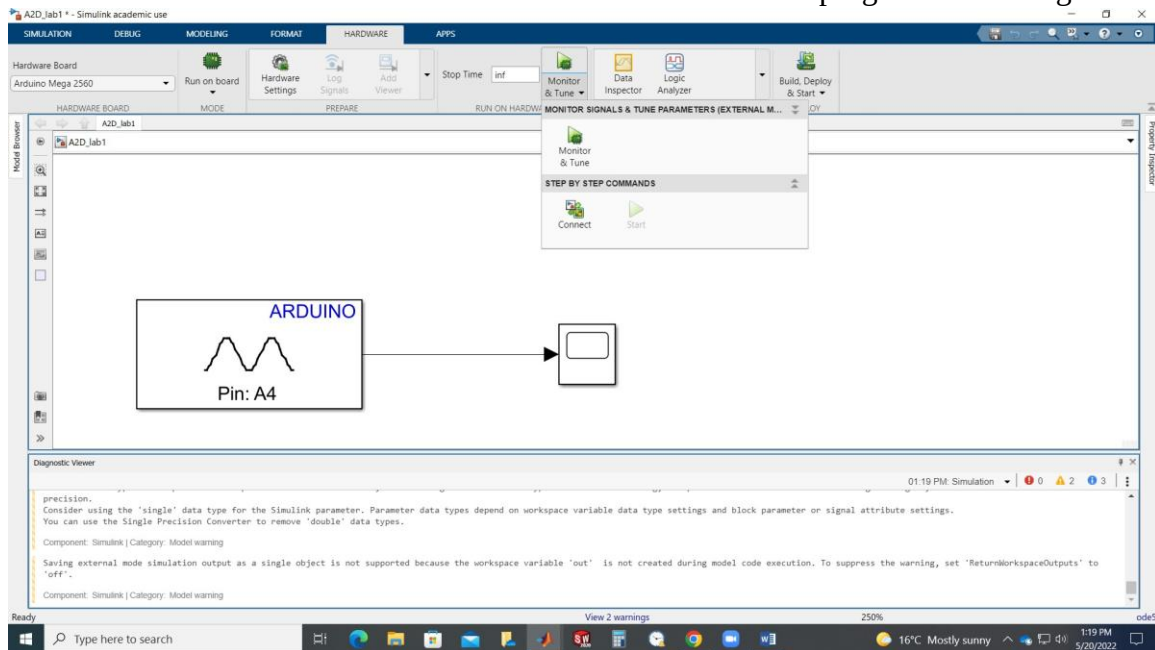


Figure 1.9: Simulation mode settings: External.

Use the “Monitor and Tune” menu to download and run the program on the target board



8. By turning the potentiometer knob, you should see the changes in the reading from analog pin A0. Read the voltage of A0 using a multimeter/oscilloscope and compare the results. Calculate how the numbers you see on the simulink scope correspond to the actual voltages on pin A0.

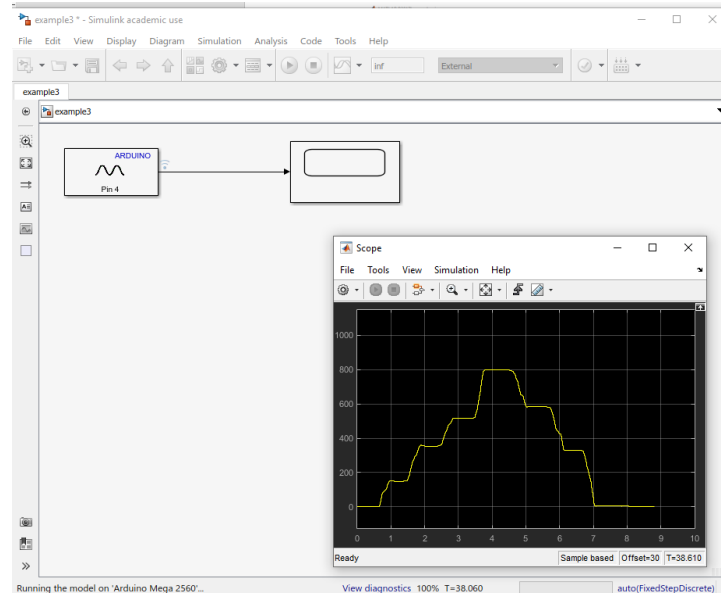


Figure 1.10: Example 1_3 results.

Example 1.4: Read digital input and display

To read a digital value from a switch and see it on a Simulink scope, do the following steps:

1. Connect the power source (Arduino 5V pin) to the normally open (N.O.) pin of the switch, then connect the COM pin to a 10 k Ω resistor and connect the other end of the resistor to GND (Arduino GND). Connect the COM to a digital input pin of the Arduino board that you want to read from (e.g., pin D8).
2. Drag and drop an “arduino digital input” block from *Simulink Support Package for Arduino Hardware*, and a “scope” and a “display” from *Sinks*. Connect the scope and display to the Arduino digital input and configure the digital input pin to the one you want to read from (e.g., pin D8) and set the sampling time to 0.01.
3. Repeat steps 4 to 7 of example 1.3 to configure the model to be deployed on the Arduino hardware.
4. Before pressing the button on the micro switch, the normally open pin is not connected to the COM pin, hence the circuit is open and the display and scope show the reading corresponding to a digital value of 0. By pressing the button, the switch connects to GND and 5V (or digital value 1) is read and shown on the display and scope.

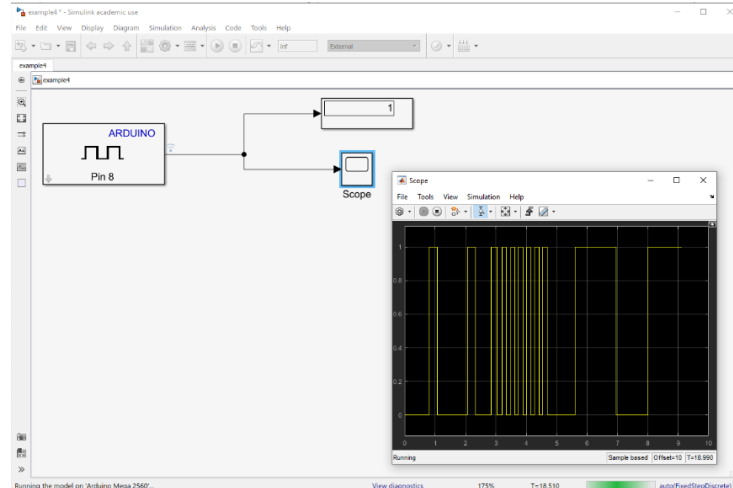


Figure 1.11: Read digital input and display.

Example 1.5: Writing digital outputs and using Simulink Dashboard elements

1. Select a digital output pin (e.g., pin D8) and connect it to the Anode (+ pin) of the LED. Connect the Cathode (- pin) of the LED to GND via a $1k\Omega$ resistor. Bring a toggle switch in SIMULINK and turn the switch ON and OFF or set the input between 0 and 1 to see the LED blink.

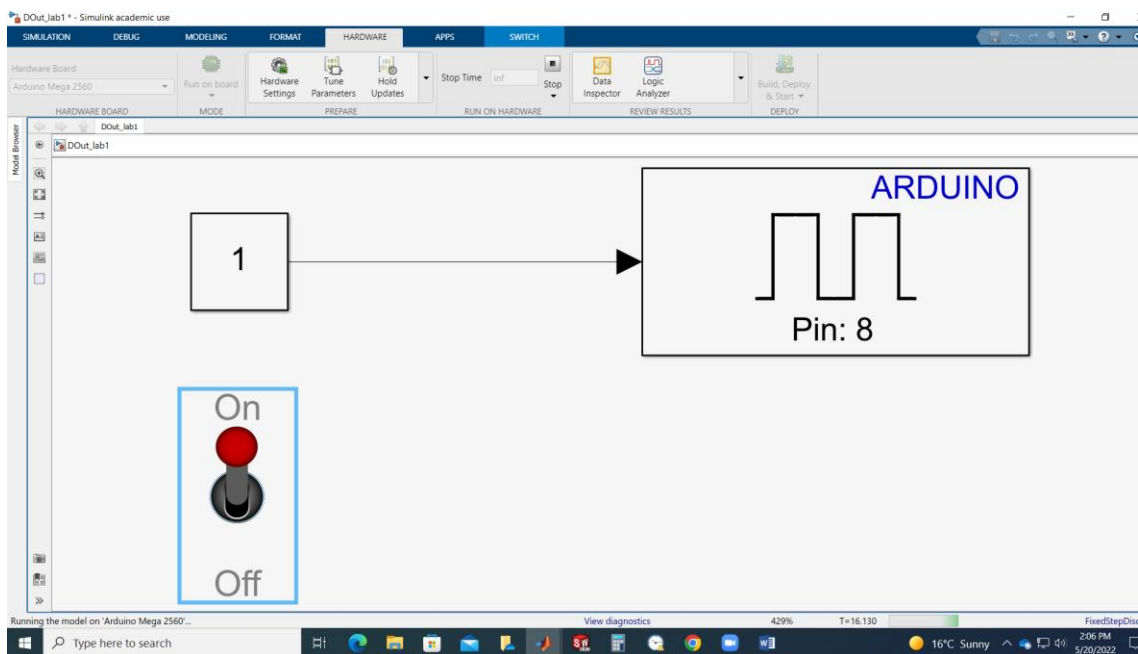


Figure 1.12: Writing digital outputs.

2. As you may have noticed, writing a digital output can only generate a digital 0 or 1 corresponding to a 0 or 5 volts signal to turn something off or on. What do you think should be done if you need to create an analog output? Can it be done directly with the Arduino board available to you? If not, what solutions do you suggest?